

Populr — engineering onboarding

Read this once before you start writing code. It takes about twenty minutes and will save you a lot of unnecessary digging later.

It's written for whoever joins next, including backend interns. It assumes you're a capable engineer who hasn't seen this codebase — not that you already know how any of it works.

1. What we're building

Populr is an AI CMO. You paste a website; it reads the business, decides what this week's marketing should be, writes the work, publishes it through accounts you connect, and reports what moved.

The bet is narrower than "AI writes your posts", and it's worth understanding before you touch the code, because most of the design follows from it:

Most marketing work isn't worth doing. The valuable act is refusing it with a reason attached — and then being able to prove the refusal was right.

Generating more is the easy half — any model does that. Keeping score of what was declined, and why, is the half nobody builds. That is why you will find so much machinery here for *not* doing things: refusal records, deterministic scoring, a planner that drops candidates on purpose. If that machinery looks disproportionate, this is the reason.

What that means for the code. The engines are deliberately built ahead of the load they will eventually take, so you'll find machinery here that is more careful than the current traffic strictly requires. That's on purpose, not over-engineering — the parts that are hard to retrofit (workspace isolation, deterministic decisions, refusal records) are the parts built first. Several questions are still genuinely open: what counts as a refusal worth recording, how a decision gets graded after the fact. You'll be asked what you think.

2. The rules that aren't negotiable

These aren't style preferences. They're the load-bearing rules — a change that breaks one gets sent back regardless of how good the rest of it is. Each exists because we got it wrong once and paid for it.

Never invent a number. Not in code, not in a prompt, not in a fixture, not in a comment. If a value wasn't measured, it renders as —, never 0. A zero on a dial and a real zero look identical and only one is true.

`lib/cmo/quality-rules.ts` carries this into every prompt; `lib/intelligence/types.ts` enforces it at a store boundary with a hard throw.

Detection is deterministic. The model only explains. An LLM never decides whether something is true, never sets a severity, never picks a verdict. It writes prose into fields we've already decided. This is what lets us say the product can't hallucinate a finding, and it's the first thing a skeptical user tests.

Say what's actually happening. If a platform can't publish, the row says so. If a trial ended, the screen says so with a way out. If we can't measure something, we say we can't. We have shipped the opposite several times and each one cost more to fix than to have avoided.

Nothing introduces itself as Gemini. The product is Populr. `lib/cmo/identity.ts` scrubs self-disclosure from model output, and `/api/cmo/respond` answers "what model are you" deterministically without calling a model at all.

3. Running it

```
git clone https://github.com/kavya5cloud/newcmo.git
cd newcmo
npm install
npm run dev           # http://localhost:3000
```

It runs with no environment variables at all. That's deliberate and it's the single most useful thing to know about this codebase. Every store is written twice — an in-memory implementation and a Neon Postgres one — behind one interface, and `db()` returns `null` when `DATABASE_URL` is unset, which selects the in-memory version:

```
export function db() {
  const url = process.env.DATABASE_URL;
  if (!url) return null;
  return neon(url);
}
```

So you get a working app, seeded data, and a full test suite without provisioning anything. Add keys only for the specific thing you're working on.

```
npm test           # vitest run - 102 files, ~1400 tests, ~20s
npx tsc --noEmit   # the type check CI runs
npm run build      # catches things tsc alone won't
```

Stack: Next.js 16.2.10 (App Router), React 19, TypeScript, Neon Postgres, Vitest, plain CSS in one file. No Tailwind, no component library, no ORM.

Two Next 16 things that will confuse you if you've used 13/14: `middleware` is `proxy.ts`, not `middleware.ts`, and `NEXT_PUBLIC_*` values are inlined at build time — changing one needs a redeploy, not a restart.

4. The shape of the code

Once you see these three patterns you can read anything here.

The repository pattern

Every store looks like this, without exception:

```
export interface ThingRepo { ... }
class InMemoryThingRepo implements ThingRepo { ... }
class NeonThingRepo implements ThingRepo { ... }

export function thingRepo(): ThingRepo {
  const sql = db();
  return sql ? new NeonThingRepo(sql) : new InMemoryThingRepo();
}
```

Callers never learn which one they got. Good examples: `lib/refusals/store.ts` (small, readable, start here), `lib/intelligence/store.ts`, `lib/learning/patterns.ts`.

Engines own their domain; nothing reaches around them

`lib/` is a set of engines, and the rule is that no agent, route or page talks to a provider, a platform SDK or a database directly. They go through the engine that owns it.

Engine	Owns
<code>lib/services/llm.ts</code>	Every model call. Provider chain, fallback, retry, caching, dedup.
<code>lib/social/</code>	Connected accounts, OAuth, publishing adapters, the publish queue.
<code>lib/content/</code>	Composition, craft scoring, the generation context.
<code>lib/agents/</code>	The nine agents, shared context, the board.
<code>lib/cmo/</code>	Classification, the deterministic planner, prompt rules, identity.
<code>lib/learning/</code>	Performance ingestion → patterns, Brand DNA.
<code>lib/market/</code>	Market research and Market Memory.
<code>lib/automation/</code>	Recurring schedules, the queue, the daily topic rotation.
<code>lib/billing/</code>	Access decisions, Polar subscriptions, webhooks.
<code>lib/refusals/</code>	What was declined and why. Read this one first.

If you find yourself importing neon outside a store, or fetching a model provider outside `lib/services/llm.ts`, stop — there's an engine for it.

Workspace keys

Everything is scoped to a workspace. Signed in, that's `"user:" + userId`; anonymous, it's `"anon:" + wsid` from the browser. `workspaceKey()` in `lib/intel.ts` is the only place that decides. Any query without

a workspace filter is a bug — we shipped one and one customer's data reached another's prompt.

5. The three paths worth tracing

Open these in order. They're most of the product.

Generation. `/api/content/compose` → `composeWithAi` (`lib/content/ai.ts`) → `assembleGenerationContext` builds one view of the business (brand voice, market, platform limits, what's already been written) → `generateText` (`lib/services/llm.ts`) walks the provider chain → `scoreDraft` (`lib/content/craft.ts`) grades it deterministically and triggers at most one rewrite → variants are cut to each platform's real character limit.

Publishing. `automationRepo` holds queue slots. Every ten minutes GitHub Actions calls `/api/cron/automation-publish`, which reclaims stalled claims, retries what's due, resolves content per slot, and hands it to the Publishing Engine with the slot id as an idempotency key. Adapters are live only when app credentials exist — otherwise a reference adapter records "published" and nothing leaves. Today LinkedIn is live; X isn't.

Planning. `/api/cmo/respond` → `classifyRequest` → `planDecision` (`lib/cmo/planner.ts`) scores candidate strategies on eight criteria, deterministically, no LLM. The winner is rendered by a model; the *decision* is not.

6. Scheduled work

What	Where	When
Publish due slots + agent sweep	<code>.github/workflows/publish.yml</code>	every 10 min
Publish reminders	<code>vercel.json</code>	daily 08:00
Outcome snapshots (GSC before/after)	<code>vercel.json</code>	Mondays 06:00

Vercel's Hobby plan allows two crons at daily granularity, both spent. Anything finer runs from GitHub Actions with `CRON_SECRET`.

A green workflow run does not mean anything published. It means the endpoint answered. Read the JSON body, not the tick. This is written on the workflow too, because it has caught people out.

7. Environment

Nothing is required, and for almost all work nothing is what you should set. The in-memory path is a complete application — seeded data, every screen, the full test suite — and it is how this codebase is meant to be developed.

There are two you might reasonably add locally:

Variable	Unlocks
GROQ_API_KEY / GEMINI_API_KEY / OPENAI_API_KEY	Real generation instead of the deterministic composer. Any one works; get your own free key rather than reusing anyone else's.
NEXT_PUBLIC_SHOW_CONTENT_ENGINE=1	Unseals the content engine (off by default). Remember NEXT_PUBLIC_* is inlined at build time.

Everything else — the database, sign-in, the social platform credentials, the token encryption key, billing, the cron secret — is deployment configuration. It is owned outside this document, you will not need it to build or test anything, and `grep` will show you the variable name at the point it is read if you are ever curious.

The rule, stated plainly: do not request, copy, paste, commit, log or configure production credentials unless you have been explicitly asked to. Not for convenience, not to reproduce a bug, not to "check something quickly". If a task genuinely seems to need one, ask first and say why — that conversation takes a minute and is always fine to have. The answer is usually that the in-memory path already covers it.

If you are given production access anyway: read if you must, never write. Assume any connection string you are handed points at production until someone tells you otherwise.

Never put a secret in a commit, a comment, a screenshot, or a chat message — **including into an AI tool**. Keys belong in `.env.local`, which is gitignored; they do not belong in a `.txt` file in the project root.

8. Definition of done

A change is finished when all four pass:

```
npx tsc --noEmit
npm test
npm run build
```

...and you have **looked at the thing you changed**. If it renders, open it. Type in it. Resize to 375px. Screenshots and computed styles beat reasoning about CSS — several bugs here typechecked, built, passed tests, and were still broken on a phone.

Tests are expected with behaviour changes. Write the test so it *fails against the old behaviour* — a test that passes either way documents nothing. If you can't make it fail, you haven't found the bug yet.

See section 9 for how the change actually gets to `main`. Short version: not by pushing to it.

9. Git and deployment

Work happens on a branch. `main` is the production branch — Vercel deploys it — so nothing reaches it except through a reviewed pull request.

```
git checkout -b intern/refusal-ledger      # or fix/..., feat/... – say what it is

npm test
npm run build

git push -u origin intern/refusal-ledger
```

Then open a PR and wait for review. **Don't push to main**, and don't merge your own PR unless you've been told to. If you find yourself typing `git push origin HEAD:main`, that's the old instruction — it's wrong now.

Two things worth being clear about, because they're easy to assume otherwise:

Access to the source is not access to production. You have the code; you do not have the database, the deployment, or any provider key, and you don't need them. Everything in section 14 is buildable and testable against the in-memory path.

Secrets never go into GitHub. Not in a commit, not in a PR description, not in an issue, not in a screenshot attached to either. Every `.env*` file is gitignored — leave it that way, and if you ever see one appear in `git status` as untracked, say so rather than committing around it. If a task looks like it needs a real credential, ask first; the answer is usually that it doesn't.

10. Traps — every one of these has actually bitten

Read this section twice. It's the most valuable page here.

Alternation binds looser than `\b`. `/\bpause|stop|hold\b/` anchors only the first and last words, so "stop" matched inside "nonstop" and "market" inside "marketing". Group your alternations.

Timezone signs. Local time is UTC *plus* the offset. Getting it backwards only breaks between 18:30 and 24:00 UTC — which is exactly when an Indian user is looking.

A cache key that ignores intent. Analysis and writing want opposite behaviour from the same cache. Analysing a page twice should give the same answer; asking for a post twice should not. `buildCacheKey` takes a salt for this reason.

Claiming work without a way to release it. The publish loop marks a slot publishing before generating. publishing only transitions to published or failed, both written by the run holding it — so a timeout stranded slots permanently and invisibly. Anything you claim, make expirable.

Sequential model calls inside a function timeout. Twenty-five slots × one generation each does not fit in sixty seconds. Budget the wall clock and yield; being killed is always worse than stopping early.

The client recomputing a server decision. The dashboard recomputed trial active from a date and overrode the server's verdict, so the API would serve a paying customer while the screen showed "your free month has ended". Servers decide; clients display.

Provider model names go stale. Groq retired both models in our chain and every request 404'd for a week. There's a `RETIRED_MODELS` guard now. A test asserting a model name passed the entire time that model was dead — assert behaviour, not configuration.

Mobile hides most of the dashboard. `.col { display: none }` and only `.col.mactive` shows. A component that copies the column markup without that class renders a black void.

Check whether it already exists. The most common mistake in this repo's history is building next to something that was already there — a duplicate workflow file, a duplicate CSS block, a second nav rail, a second do-not-repeat mechanism. `grep` before you create.

11. Working with AI-assisted development

We use Claude Code and similar tools heavily, and we want you to. This section exists because the way you use them is the difference between shipping faster and shipping something nobody understands.

The original architecture here — the engines, the repository pattern, the deterministic planner — was designed and written by hand. As the surface has grown we increasingly use AI tools for implementation, exploration, debugging, tests, refactors, docs and finding your way around a codebase you haven't read. That's deliberate. A lot of what exists now would not exist yet otherwise.

The expectation is not *"ask Claude to write the feature."* It is:

Use AI to move faster while you understand and own the result.

That distinction is the whole section. Concretely:

- **Generated code is a draft, not an answer.** Read it before you accept it. If you can't explain what a block does, you're not finished — ask the model to explain it, then check the explanation against the code.
- **Verify the claims.** Models state things about a codebase confidently and are sometimes wrong about this one. `grep` for the function it says exists. Open the file.
- **Run the four checks yourself** (section 8). "It compiled in the chat" is not a check.
- **Open the app when the change is visible.** Several bugs here typechecked, built, passed tests, and were still broken at 375px. A screenshot settles it; reasoning about CSS does not.
- **Push back on architecture suggestions.** A plausible refactor that puts a `fetch` to a model provider outside `lib/services/llm.ts` is wrong here, however clean it looks. You know the constraints in this document; the model is inferring them.
- **When the tool stalls, think.** Models get stuck in loops, re-suggest the thing that already failed, or confidently paper over a root cause. That's your cue to read the code and reason it through — and sometimes to write the fix by hand. That is a normal outcome, not a failure.
- **Grep before you generate.** Section 10's last trap is the one these tools make easiest to fall into: asked for something, a model will happily write a second implementation of a thing you already have, and it will look perfectly reasonable.

The failure mode we care about isn't "used AI". It's a change nobody can explain, defend or debug. Whoever opened the PR owns its correctness — the tooling doesn't transfer that.

AI is a multiplier, not a substitute for engineering understanding. We don't want engineers who avoid these tools. We want engineers who are dramatically more effective because they know how to use them well, and who can tell when the tool is wrong.

12. What we expect from a Backend Intern

You're joining an engineering team. The work is building and running a production system — routes, stores, queues, background work — that happens to have models in one layer of it.

- **Understand before you modify.** Trace the request, find the engine that owns it, read the existing tests. Then change something.
 - **Be comfortable reading unfamiliar code.** You will not be given a tour of every file. Being able to open a 400-line module you've never seen and work out what it does is most of the job.
 - **Ask about architecture.** Particularly about section 2. If a rule looks wrong, say so — that's a conversation we want, not a challenge to authority.
 - **Think in failure modes.** What happens on a second click, a timeout, a retry, a duplicate webhook, a lambda that dies mid-write? Most of the traps in section 10 are answers to that question that we learned the hard way.
 - **Know what is safe to repeat.** Reading is. Writing usually isn't. Anything that spends money or publishes needs a key that makes the second attempt a no-op.
 - **Write regression tests.** Section 8's rule holds: write it so it fails against the old behaviour.
 - **Debug independently.** Read the error. Reproduce it. Find the cause before you change anything. "It works now" without knowing why is not a fix.
 - **Know which side of the line you're on.** Deterministic logic and model-driven behaviour are treated completely differently here. Be clear at all times about which one you're changing.
 - **Care about latency and cost.** Every model call is money and seconds; every interval in a browser tab is an invocation. Both show up on a bill.
 - **Verify your own work.** Section 8's definition of done, every time — including looking at it. Take the change all the way to "I have seen this working", not "it should work".
-

13. Backend ideas worth being comfortable with

Practical list, grounded in what's actually here. You don't need all of it on day one; you will meet all of it.

API routes and their boundaries Every route in `app/api/` follows the same shape: authenticate, rate-limit, validate the body, resolve a workspace, call an engine, return JSON. `lib/throttle.ts` holds the limiter. The validation is deliberately explicit rather than schema-driven — read one route end to end (`app/api/content/compose/route.ts`) and you have read all of them.

Data access The repository pattern in section 4 is the whole story: an interface, an in-memory implementation, a Neon one, and `db()` choosing between them. Tables are created by `ensure*` guards behind `RUNTIME_DDL` rather than a migration tool. Understand why every query carries a workspace filter before you write one — section 4's last paragraph is not theoretical.

Queues, workers and background execution `lib/jobs/` is a full engine: `QueueManager` holds priority entries, `WorkerPool` drains them with a concurrency limit, a timeout and a dead-letter queue, and `JobEngine` walks each job through its stage machine. Read `worker.ts` first; it is short and it is the part with the re-entrancy guard.

The thing to understand deeply is what *doesn't* survive: that queue is in-memory and per-process. On a serverless platform a job created by one invocation is invisible to the next, which is why durable work is keyed in Postgres and why `lib/creative/jobs.ts` exists alongside the engine rather than inside it. The comment at the top of that file explains the decision.

Retries, idempotency and concurrency `lib/services/llm.ts` classifies upstream failures (`classifyUpstream`, `isTransientStatus`) so a 429 is retried and a 400 is not. Idempotency shows up three ways here, and they're worth comparing: a `UNIQUE` constraint with `ON CONFLICT DO NOTHING` in `creative_jobs`, a slot id passed to the publishing engine, and an in-flight ref in the composer that stops a double-click. Claiming work is the related trap — anything you mark as "in progress" needs an expiry, or a timeout strands it forever (section 10).

Security Workspace isolation is the one that matters most; everything else follows from section 2 and section 9. Provider keys are server-side only and never reach a client bundle. Stored OAuth tokens are encrypted at rest. Nothing accepts a user-supplied URL and fetches it — if you ever add something that does, that is an SSRF surface and it needs an allowlist.

Performance and cost Two different things get billed: how long a function is alive, and how much CPU it actually burns. A `setTimeout` inside a handler costs duration; a `JWT verify` costs CPU. On the client, an unbounded `setInterval` is a request generator — one tab polling every second is 3,600 invocations an hour whether or not anyone is looking at it. Budget wall-clock inside cron handlers; being killed at the timeout is always worse than stopping early.

Observability Logging here is `console.info(JSON.stringify({ event, ...fields }))` — structured, greppable in the Vercel log view, and deliberately free of secrets and prompt text. Jobs additionally emit events through `JobEventBus` and append to a store, which is what the execution dashboard reads. When you add a code path that can fail silently, add an event for it.

Where models fit Only one layer talks to a provider (`lib/services/llm.ts`), and it handles the chain, fallback, retry and caching so nothing else has to. Around it sits ordinary backend work: `lib/llm-json.ts` parses structured output that arrives wrapped in prose or truncated; `lib/content/craft.ts` grades the result deterministically; `lib/cmo/planner.ts` makes the actual decision with no model involved. Treat a model call as an unreliable network dependency that returns prose, and the rest of the design follows.

A note on scope: we do not train or fine-tune models, we have no labelled dataset, and there is no evaluation harness beyond the test suite. If you read otherwise somewhere, it isn't here yet.

14. A way through it

Rough progression rather than a schedule. Most of it is reading.

First, the product. Run it with no environment variables, paste a website, and use it as a user would. Then read `lib/refusals/` end to end — about 370 lines, and it explains the philosophy better than section 1 does.

Then the three paths in section 5. Generation, publishing, planning. Follow each one from the route to the store with the files open. You're looking for where the workspace key enters, which engine owns each step, and where the decision actually gets made.

Then where the model enters. Re-walk generation and mark four points: where context is assembled, where the prompt is built, where the model is called, and where the output is validated or graded. Notice how much deterministic code sits either side of the one model call. That shape is the architecture.

Then how we know it works. Read the tests for the paths you traced — `tests/content-craft.test.ts`, `tests/craft-locale.test.ts`, and `tests/sarvam-provider.test.ts` for provider behaviour and failure handling. Look particularly at how a provider outage is tested without a provider being involved.

Then own something small and measurable. One of the tasks below. Small, real, with tests.

15. Where to start

Assuming you've been through section 14, here are real first tasks, in rough order of size.

Before you write any of them, do this. It isn't ceremony — it's the sequence that stops you from building a second copy of something that already exists, which is this repo's most common historical mistake:

1. **Trace the path.** Route → engine → store. Have the files open.
2. **Name the owner.** Which engine in the section 4 table owns this behaviour? The change belongs there, not in the route and not in the component.
3. **Find the existing tests.** `grep` the function name in `tests/`. They tell you what the current behaviour is *supposed* to be.
4. **Explain the current behaviour in your own words** — in the PR description, an issue, or a message to whoever is reviewing. If you can't, you're not ready to change it.
5. **Say where the change goes** and why there.
6. **Then implement it.**
7. **Add or update tests** so they fail against the old behaviour.
8. **Verify** — the four checks in section 8, plus looking at it if it renders.

That's understand → plan → implement → test → verify. It is deliberately not prompt → copy → PR, and the difference shows up immediately in review.

Task A — wire the refusal recorder into the planner. `refusalsFromPlan()` converts a planning pass into recorded refusals. It is exported and referenced nowhere, so nothing has ever written to the refusal store.

The planner already scores candidates, keeps three and drops the rest — those dropped candidates *are* the refusals. Small change, real tests, and it puts the first real row in a table the whole design is built around.

Task B — the grading pass. Extend the weekly outcome job to resolve refusals whose checkable window has passed, from measured data only. unknown must stay a normal, permanent, honest state for most of them. This is the piece we most want someone to own.

Task C — `llms.txt`. A route that tells AI search engines what this site is. Genuinely missing, genuinely cheap, and it belongs to the SEO agent.

Ask before you refactor something large. Ask *immediately* if you think a rule in section 2 is wrong — those are load-bearing, and the reasoning behind each one is a story we'd rather tell you than have you rediscover.